



Technical reference manual
1.0.0

EtherBench

Programming Reference Manual

Complete technical API documentation for the EtherBench platform. Includes SCPI sub-systems syntax and direct low-level serial shell interactions.

Automated actions controls

actions

- SCPI Syntax: ACTIONS
- SCPI Syntax: ACTIONS?

Fast (<100µs) Non-Blocking

Configure a new action to be performed on press of the ACTx buttons.

ARGUMENTS / RETURNS

Argument	Description
ID	int<1,2,3>
ACTION	str

Returns	Description
STATUS	int<0...>

ADDITIONAL INFORMATION

WARNING

Bount actions aren't checked, and are interpreted as any other command. Therefore, ACTx buttons may be used as reset.

BUS TRANSFER EXAMPLES

Command string: ACTION 1,*IDN

Device transmission: 0

Analog Subsystem Metrology (ADC / DAC)

analog

- SCPI Syntax: `ANALOG`
- SCPI Syntax: `ANALOG?`
- Shell Hook: `analog`

Very slow

Non-Blocking

Perform an IO operation over the analog subsystem. Query target the ADC, Set target the DAC.

ARGUMENTS / RETURNS

Argument	Description
<code>SOURCE</code>	integer<0,1>
<code>VALUE</code>	str<3.19V...>

Returns	Description
<code>STATUS</code>	int<0...>
<code>[VALUE]</code>	str

BUS TRANSFER EXAMPLES

Command string: `ANALOG[?] 1, [3.19V]`

Device transmission: `0[ , 3.28V]`

Configuration Sub-System

configuration.voltage

- SCPI Syntax: CONFIGURATION:VOLTAGE
- SCPI Syntax: CONFIGURATION:VOLTAGE?
- Shell Hook: config

Slow Non-Blocking

Configure the IO voltage to be used.
Query read back the current voltage.
Set configure a new voltage.

ARGUMENTS / RETURNS

Argument	Description
[VALUE]	str<3.3V>

Returns	Description
STATUS	int<0...>
[VALUE]	<str<2.8V>

ADDITIONAL INFORMATION

WARNING
The configuration range is limited by onboard EEPROM, preventing from any damages.

BUS TRANSFER EXAMPLES

Command string: CONFIGURATION:VOLTAGE[?] [2.85V]

Device transmission: 0, [2.78V]

Hardware Input Outputs

hardware.can

- SCPI Syntax: `HARDWARE:CAN`
- SCPI Syntax: `HARDWARE:CAN?`
- Shell Hook: `io`

Slow **Blocking**

Perform an IO operation over the CAN bus. Query mean read, Set mean write

ARGUMENTS / RETURNS

Argument	Description
<code>[PAYLOAD]</code>	<code>int(s),<0x00--0xFF></code>

Returns	Description
<code>STATUS</code>	<code>int<0...></code>
<code>[PAYLOAD]</code>	<code>int(s),<0x00--0xFF></code>

ADDITIONAL INFORMATION

WARNING

As this bus is asynchronous, the read command may send back a LOT of data, capped to 1024 bytes (reception memory). If speed is a requirement, consider using the raw UDP streaming.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:CAN[?] [ , 0x487643 ]`

Device transmission: `0, [ 0x7865456 ]`

hardware.i2c

- SCPI Syntax: `HARDWARE:I2C`
- SCPI Syntax: `HARDWARE:I2C?`
- Shell Hook: `io`

Slow Blocking

Perform an IO operation over the I2C bus. Query mean read, Set mean write

ARGUMENTS / RETURNS

Argument	Description
[ADDRESS]	int<0x00--0xFF>
[PAYLOAD]	int(s),<0x00--0xFF>
[EXTENDED]	int<0,1>
[SPEED]	str<100kHz,400kHz,1MHz>

Returns	Description
STATUS	int<0...>
[PAYLOAD]	int(s),<0x00--0xFF>

ADDITIONAL INFORMATION

WARNING

This IO operation feature a single operation. Some required back to back operation, thus emit two IO operation.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:I2C[?] [0x78][,0x487643]`

Device transmission: `0,[0x7865456]`

hardware.spi

- SCPI Syntax: `HARDWARE:SPI`
- SCPI Syntax: `HARDWARE:SPI?`
- Shell Hook: `io`

Slow Blocking

Perform an IO operation over the SPI bus. Query mean read, Set mean write. Address is the current selected slave.

ARGUMENTS / RETURNS

Argument	Description
<code>[ADDRESS]</code>	<code>int<1--2></code>
<code>[PAYLOAD]</code>	<code>int(s),<0x00--0xFF></code>

Returns	Description
<code>STATUS</code>	<code>int<0...></code>
<code>[PAYLOAD]</code>	<code>int(s),<0x00--0xFF></code>

ADDITIONAL INFORMATION

WARNING

This IO operation feature a single operation. Some required back to back operation, thus emit two IO operation.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:SPI[?] 1[ , 0x487643 ]`

Device transmission: `0, [ 0x7865456 ]`

hardware.spy.can

- SCPI Syntax: `HARDWARE : SPY : CAN`
- Shell Hook: `spy`

Fast (<100µs)

Non-Blocking

Configure the spy module, to look over an CAN bus. The bus is then placed as a slave, and will listen for anything.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>STATUS</code>	int<0...>

ADDITIONAL INFORMATION

WARNING

The reception buffer is capped to 1024 bytes (reception memory). Use the UDP streaming for faster transfers.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE : SPY : CAN`

Device transmission: `0`

hardware.spy.i2c

- SCPI Syntax: `HARDWARE:SPY:I2C`
- Shell Hook: `spy`

Fast (<100µs)

Non-Blocking

Configure the spy module, to look over an I2C bus. The bus is then placed as a slave, and will listen for anything.

ARGUMENTS / RETURNS

Argument	Description
[ADDRESS]	int<0x00--0x3FF>
[EXTENDED]	int<0,1>

Returns	Description
STATUS	int<0...>

ADDITIONAL INFORMATION

WARNING

The reception buffer is capped to 1024 bytes (reception memory). Use the UDP streaming for faster transfers.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:SPY:I2C`

Device transmission: `0`

hardware.spy.spi

- SCPI Syntax: `HARDWARE : SPY : SPI`
- Shell Hook: `spy`

Fast (<100µs) **Non-Blocking**

Configure the spy module, to look over an SPI bus. The bus is then placed as a slave, and will listen for anything.

ARGUMENTS / RETURNS

Argument	Description
<code>[ADDRESS]</code>	<code>int<1--2></code>

Returns	Description
<code>STATUS</code>	<code>int<0...></code>

ADDITIONAL INFORMATION

WARNING

The reception buffer is capped to 1024 bytes (reception memory). Use the UDP streaming for faster transfers.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE : SPY : SPI`

Device transmission: `0`

hardware.spy.uart

- SCPI Syntax: `HARDWARE : SPY : UART`
- Shell Hook: `spy`

Fast (<100µs) **Non-Blocking**

Configure the spy module, to look over an UART bus. The bus is then placed as a slave, and will listen for anything.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>STATUS</code>	int<0...>

ADDITIONAL INFORMATION

WARNING

The reception buffer is capped to 1024 bytes (reception memory). Use the UDP streaming for faster transfers.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE : SPY : UART`

Device transmission: `0`

hardware.uart

- SCPI Syntax: `HARDWARE:UART`
- SCPI Syntax: `HARDWARE:UART?`
- Shell Hook: `io`

Slow Blocking

Perform an IO operation over the UART bus. Query mean read, Set mean write

ARGUMENTS / RETURNS

Argument	Description
<code>[PAYLOAD]</code>	<code>int(s),<0x00--0xFF></code>

Returns	Description
<code>STATUS</code>	<code>int<0...></code>
<code>[PAYLOAD]</code>	<code>int(s),<0x00--0xFF></code>

ADDITIONAL INFORMATION

WARNING

As this bus is asynchronous, the read command may send back a LOT of data, capped to 1024 bytes (reception memory). If speed is a requirement, consider using the raw UDP streaming.

BUS TRANSFER EXAMPLES

Command string: `HARDWARE:UART[?] [0x487643]`

Device transmission: `0, [0x7865456]`

Logging controls

logger.subscribe

- SCPI Syntax: `LOGGER:SUBSCRIBE`
- Shell Hook: `logger`

Very fast (<10µs)

Non-Blocking

Connect a log source (spy, serial bus, Filesystem, UDP Stream ...) to a sink (Filesystem, UDP Stream...).

ARGUMENTS / RETURNS

Argument	Description
<code>SOURCE</code>	<code>str<...></code>
<code>TARGET</code>	<code>str<...></code>

Returns	Description
<code>STATUS</code>	<code>int<0...></code>
<code>ID</code>	<code>int<0...></code>

BUS TRANSFER EXAMPLES

Command string: `LOGGER:SUBSCRIBE /serial/usart0,/network/udp`

Device transmission: `0,19`

logger.unsubscribe

- SCPI Syntax: `LOGGER:UNSUBSCRIBE`

Very fast (<10µs)

Non-Blocking

Disconnect a link.

ARGUMENTS / RETURNS

Argument	Description
ID	int<0...>

Returns	Description
STATUS	int<0...>

BUS TRANSFER EXAMPLES

Command string: `LOGGER:UNSUBSCRIBE 19`

Device transmission: `0`

Target Programming Interface

programmation.connect

- SCPI Syntax: PROGRAMMATION:CONNECT
- Shell Hook: programmer

Very slow Blocking

Connect the DUT over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
BUS	str

Returns	Description
STATUS	int<0...>

ADDITIONAL INFORMATION

IMPORTANT

This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: PROGRAMMER:CONNECT?

Device transmission: 0

programming.detect

- SCPI Syntax: PROGRAMMATION:DETECT
- Shell Hook: programmer

Very slow Blocking

Detect if the DUT is present over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
BUS	str

Returns	Description
STATUS	int<0...>
RESULT	int<0(yes),1(no)>

ADDITIONAL INFORMATION

IMPORTANT

This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: PROGRAMMER:DETECT?

Device transmission: 0,0

programming.device

- SCPI Syntax: PROGRAMMATION:DEVICE
- Shell Hook: programmer

Very slow Blocking

Read the DUT descriptions values over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
BUS	str

Returns	Description
STATUS	int<0...>
VALUES	int[]<0x98765432,...>

ADDITIONAL INFORMATION

IMPORTANT

This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: PROGRAMMER:DEVICE?

Device transmission: 0, 0x98765432

programming.flash

- SCPI Syntax: PROGRAMMATION:FLASH
- Shell Hook: programmer

Very slow Blocking

Flash the DUT over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
BUS	str
FILE	str<...>

Returns	Description
STATUS	int<0...>

ADDITIONAL INFORMATION

IMPORTANT

This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: PROGRAMMER:FLASH /sd/firmware.bin

Device transmission: 0

programming.read

- SCPI Syntax: PROGRAMMATION:READ
- Shell Hook: programmer

Very slow

Blocking

Read the DUT memory over the SWD/JTAG buses. Push the data over the standard logger source

ARGUMENTS / RETURNS

Argument	Description
BUS	str

Returns	Description
STATUS	int<0...>

ADDITIONAL INFORMATION

IMPORTANT

This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: PROGRAMMER:READ?

Device transmission: 0

programming.status

- SCPI Syntax: PROGRAMMATION:STATUS
- Shell Hook: programmer

Very slow Blocking

Check if the DUT is still running over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
BUS	str

Returns	Description
STATUS	int<0...>
RESULT	int<0(yes), 1 (false)>

ADDITIONAL INFORMATION

IMPORTANT

This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: PROGRAMMER:STATUS?

Device transmission: 0, 0

programming.verify

- SCPI Syntax: PROGRAMMATION:VERIFY
- Shell Hook: programmer

Very slow Blocking

Verify the flashed binaries over the SWD/JTAG buses.

ARGUMENTS / RETURNS

Argument	Description
BUS	str

Returns	Description
STATUS	int<0...>
RESULT	int<0 (sucess),1 (error)>

ADDITIONAL INFORMATION

IMPORTANT

This command is built on CPU polled interfaces. No other operation will work when running it.

BUS TRANSFER EXAMPLES

Command string: PROGRAMMER:VERIFY? /sd/firmware.bin

Device transmission: 0,0

Bus scanning

scan.can

- SCPI Syntax: `SCAN:CAN?`
- Shell Hook: `scan`

Very slow **Blocking**

Scan the CAN bus for any active device on it.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>STATUS</code>	int<0...>
<code>RESULT</code>	int<0(yes),1(no)>

BUS TRANSFER EXAMPLES

Command string: `SCAN:CAN?`

Device transmission: `0,0`

scan.i2c

- SCPI Syntax: `SCAN:I2C?`
- Shell Hook: `scan`

Very slow

Blocking

Scan the I2C bus for any active device on it.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>STATUS</code>	int<0...>
<code>RESULT</code>	int<0(yes),1(no)>

BUS TRANSFER EXAMPLES

Command string: `SCAN:I2C?`

Device transmission: `0,0`

scan.spi

- SCPI Syntax: `SCAN:SPI?`
- Shell Hook: `scan`

Very slow

Blocking

Scan the SPI bus for any active device on it.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>STATUS</code>	int<0...>
<code>RESULT</code>	int<0(yes),1(no)>

BUS TRANSFER EXAMPLES

Command string: `SCAN:SPI?`

Device transmission: `0,0`

scan.uart

- SCPI Syntax: `SCAN:UART?`
- Shell Hook: `scan`

Very slow

Blocking

Scan the UART bus for any active device on it.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>STATUS</code>	int<0...>
<code>RESULT</code>	int<0(yes),1(no)>
<code>[BUS]</code>	int<0,1,2>

BUS TRANSFER EXAMPLES

Command string: `SCAN:UART?`

Device transmission: `0,0,1`

Sequencing

sequences.add

- SCPI Syntax: SEQUENCES:ADD

Fast (<100µs)

Non-Blocking

Add an instruction on the sequence. Instruction will be 'compiled'.

ARGUMENTS / RETURNS

Argument	Description
NAME	str<...>
INSTRUCTION	str<...>

Returns	Description
STATUS	int<0...>

ADDITIONAL INFORMATION

WARNING

For complex sequences, consider transferring it as a whole over the FTP server.

BUS TRANSFER EXAMPLES

Command string: SEQUENCE:ADD flasher execute *IDN

Device transmission: 0

sequences.get

- SCPI Syntax: SEQUENCES:GET?

Slow Blocking

Send the binary file that correspond to the sequence

ARGUMENTS / RETURNS

Argument	Description
NAME	str<...>

Returns	Description
STATUS	int<0...>
DATA	<int[]<0x98,0x76...>

BUS TRANSFER EXAMPLES

Command string: SEQUENCE:GET

Device transmission: 0,0x98,0x87...

sequences.new

- SCPI Syntax: SEQUENCES:NEW

Fast (<100µs) Non-Blocking

Add a new sequence on the device

ARGUMENTS / RETURNS

Argument	Description
NAME	str<...>

Returns	Description
STATUS	int<0...>

BUS TRANSFER EXAMPLES

Command string: SEQUENCE:NEW flasher

Device transmission: 0

sequences.run

- SCPI Syntax: SEQUENCES:RUN

Slow

Non-Blocking

Execute the sequence.

ARGUMENTS / RETURNS

Argument	Description
NAME	str<...>

Returns	Description
STATUS	int<0...>
PASS	int<0-...>
FAIL	int<0-...>

BUS TRANSFER EXAMPLES

Command string: SEQUENCE:RUN flasher

Device transmission: 0,1,0

IEEE-488.2 Standard Core Commands

*cls

- SCPI Syntax: *CLS

Very fast (<10µs)

Blocking

Clear all the SCPI exposed registers on the device.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
STATUS	int<0...>

ADDITIONAL INFORMATION

WARNING

This does not perform a reset of the device. Hence, statuses may be retained.

BUS TRANSFER EXAMPLES

Command string: *CLS

Device transmission: 0

***ese**

- SCPI Syntax: *ESE
- SCPI Syntax: *ESE?

Very fast (<10µs)

Blocking

Set bits on the Event Status Register. Returns the event status mask.

ARGUMENTS / RETURNS

Argument	Description
XX	Bits to be set (if maskable).

Returns	Description
STATUS	int<0...>
MASK	int<-1-255>

ADDITIONAL INFORMATION

Bit signification

Bit	Name	Signification
0	OPC	Operation complete
1		Unused
2	QYE	The device could not process a command. Buffers are perhaps full ?
3	DDE	Self test failed
4	EXE	A command failed it's execution
5	CMD	An invalid command was received
6		Unused
7		Unused

BUS TRANSFER EXAMPLES

Command string: `*ESE 255`

Device transmission: `0,255`

***idn**

- SCPI Syntax: `*IDN?`

Very fast (<10µs) **Blocking**

Return the identification string for the device.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>STATUS</code>	<code>int<0...></code>
<code>IDN</code>	

BUS TRANSFER EXAMPLES

Command string: `*IDN?`

Device transmission: `0,EtherBenchv1:[build-date]:[firmware-version]`

***opc**

- SCPI Syntax: *0PC
- SCPI Syntax: *0PC?

Very fast (<10µs)

Blocking

Check for current running commands

ARGUMENTS / RETURNS

Argument	Description
XX	Value

Returns	Description
STATUS	int<0...>

BUS TRANSFER EXAMPLES

Command string: *0PC?

Device transmission: 0,0K

***opt**

- SCPI Syntax: `*OPT`

Very fast (<10µs)

Blocking

Returns the options string of the device.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>STATUS</code>	int<0...>

BUS TRANSFER EXAMPLES

Command string: `*OPT?`

Device transmission: `0,EtherBenchv1:[]`

***rst**

- SCPI Syntax: `*RST`

Slow **Blocking**

Reset the device, equivalent to a power on-off-on cycle.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>STATUS</code>	int<0...>

ADDITIONAL INFORMATION

WARNING

Perform an hard reset of the device. Any openned sessions will be **brutally** closed.

BUS TRANSFER EXAMPLES

Command string: `*RST`

Device transmission: `0`

***sre**

- SCPI Syntax: *SRE
- SCPI Syntax: *SRE?

Very fast (<10µs)

Blocking

Return the service query status.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
STATUS	int<0...>

ADDITIONAL INFORMATION

NOTE

This command is implented as a placeholder. Always return false.

BUS TRANSFER EXAMPLES

Command string: *SRE?

Device transmission: 0

***stb**

- SCPI Syntax: `*STB?`

Very fast (<10µs)

Blocking

Read the status byte register

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>STATUS</code>	int<0...>

ADDITIONAL INFORMATION

NOTE

This command is implented as a placeholder. Always return 0.

BUS TRANSFER EXAMPLES

Command string: `*STD?`

Device transmission: `0`

***tst**

- SCPI Syntax: *TST?

Very fast (<10µs) Blocking

Perform the self-test of the device. Check that all threads are running.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
STATUS	int<0...>

BUS TRANSFER EXAMPLES

Command string: *TST?

Device transmission: 0

***wai**

- SCPI Syntax: *WAI

Slow Blocking

Wait for all operations to be executed.

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
STATUS	int<0...>

BUS TRANSFER EXAMPLES

Command string: *WAI

Device transmission: 0

POSIX Filesystem & Shell Commands

cat

- Shell Hook: `cat`

Normal

Blocking

Show a file from the filesystem into the current terminal

ARGUMENTS / RETURNS

Argument	Description
<code>file</code>	str

Returns	Description
<code>CAT</code>	str

BUS TRANSFER EXAMPLES

Command string: `cat /sd/hello.txt`

Device transmission: `Hello World !`

cd

- Shell Hook: cd

Fast (<100µs) Blocking

Change the current directory

ARGUMENTS / RETURNS

Argument	Description
target	str

Returns	Description
CD	OK

BUS TRANSFER EXAMPLES

Command string: cd /sd

Device transmission: ``

clear

- Shell Hook: clear

Very fast (<10µs) Blocking

Clear the current terminal

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
RST	OK

BUS TRANSFER EXAMPLES

Command string: clear

Device transmission: ``

ls

- Shell Hook: `ls`

Normal Blocking

List files on the current directory

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>LS</code>	...

BUS TRANSFER EXAMPLES

Command string: `ls`

Device transmission: `hello.txt`

mkdir

- Shell Hook: `mkdir`

Fast (<100µs) Blocking

Create a directory on the current location

ARGUMENTS / RETURNS

Argument	Description
<code>NAME</code>	str

Returns	Description
<code>MKDIR</code>	OK

BUS TRANSFER EXAMPLES

Command string: `mkdir /sd/test/`

Device transmission: ``

mv

- Shell Hook: mv

Slow Blocking

Move a file from a location to another one

ARGUMENTS / RETURNS

Argument	Description
SOURCE	str
TARGET	str

Returns	Description
MV	OK

BUS TRANSFER EXAMPLES

Command string: mv /sd/hello.txt /sd/hello2.txt

Device transmission: ``

pwd

- Shell Hook: `pwd`

Normal **Blocking**

Print the current location on the filesystem

ARGUMENTS / RETURNS

This command does not take any arguments.

Returns	Description
<code>PWD</code>	str

BUS TRANSFER EXAMPLES

Command string: `PWD`

Device transmission: `/sd/example/`